

云天励飞 26Q2 定制需求方案

1. 支持 SoC 密钥轮换

1.1 原始需求

支持 SoC 密钥轮换，当下游厂商不信任上游厂商，或者发生 SoC 密钥泄漏时，支持在 TEST/DEV/MANU 生命周期下，通过 OTP bitmap 更换 SoC key，具体需求如下：

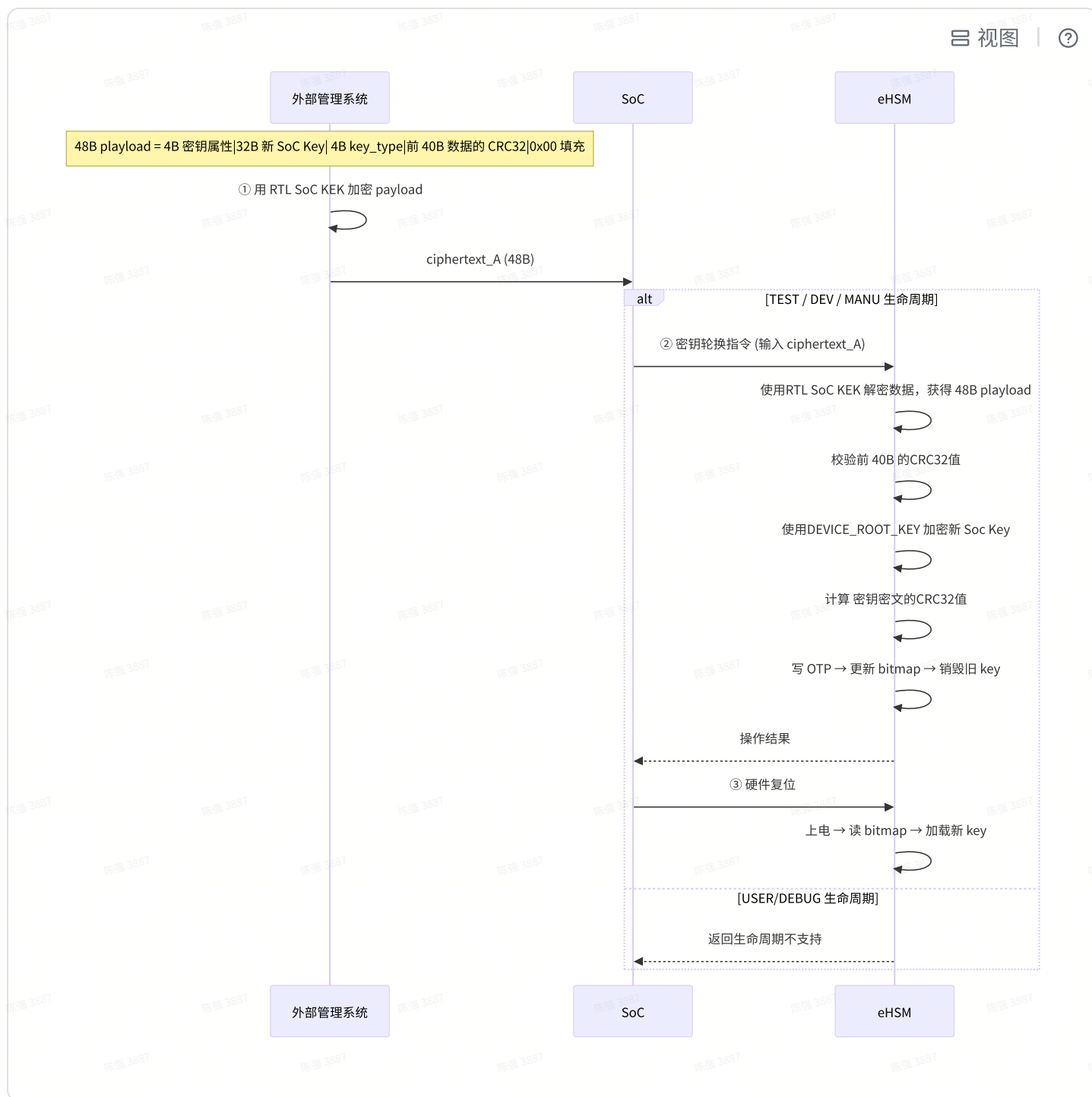
- 仅支持 3 个 SoC key 的轮换，每个 SoC key 支持仅一次轮换
- 密钥轮换流程
 - 无效化（废弃）当前旧 SoC key
 - 烧录新 SoC key
 - 通过 OTP bitmap 修改密钥映射表指向新 SoC key
- 后续使用新 SoC key

1.2 具体方案

1.2.1 改动概述

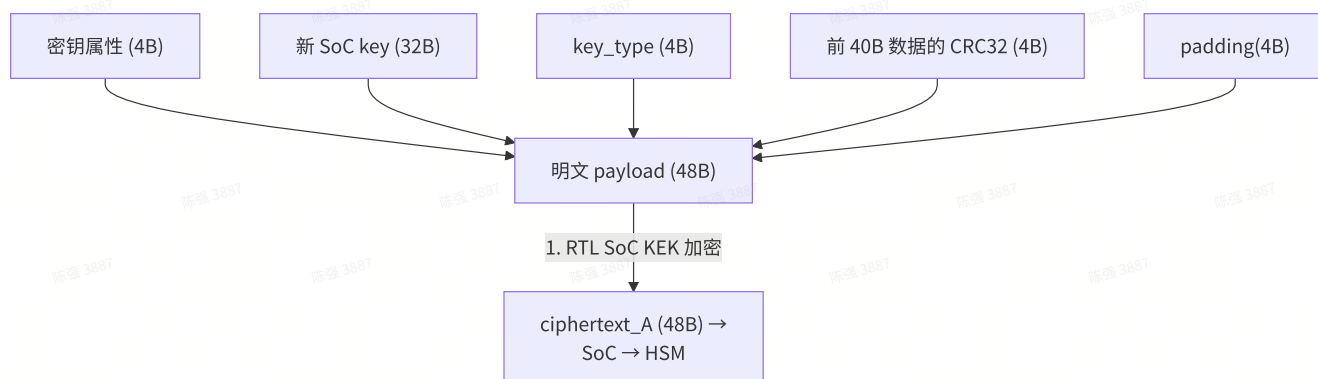
- **新增 OTP bitmap**（1 字节，HSM 内部管理），每 bit 控制一个 SoC key 的原始/轮换状态
- **新增专用 Mailbox 指令**，在 TEST/DEV/MANU 生命周期支持 SoC key 写入 OTP。Host 只需指定要替换的 key 类型，HSM 内部自动管理 bitmap
- 支持的 3 个轮换 key（具体类型和复用策略以客户确认为准）：
 - SoC Verify Key（启动验证 + 升级验证复用）
 - SoC Encrypt Key（启动解密 + 升级解密复用）
 - SoC Debug Key（调试鉴权）

1.2.2 端到端流程



流程说明:

1. 外部管理系统准备 48 字节密文



- 拼接 payload: 密钥属性 (4B) + 新 SoC key 明文 (32B) + `key_type` (4B) + 前 40B 数据的 CRC32 (4B) + 0x00 padding (4B) → 48 字节 payload
 - `key_type` 指定要替换哪个 key (0=Verify / 1=Encrypt / 2=Debug)
 - 前 40B 数据的 CRC32 (4B) 是为了保证传输过程中数据的正确性, 防止数据错误以及被篡改
- 用 RTL SoC KEK 加密 48B payload → `ciphertext_A`
 - 这一步是为了保证终端无法直接获得写入 OTP 的明文数据, 只有密文

2. SoC 调用密钥轮换指令 (或 Host API)

- SoC 将 `ciphertext_A` (48 字节) 通过新增 Mailbox 指令 (或 Host API) 发送给 HSM
- HSM 完成如下操作
 - 检查生命周期, 如果是 USER/DEBUG 生命周期, 返回生命周期不支持的错误码
 - HSM 使用 RTL SoC KEK 解密 → 校验密钥属性一致性 → 获得密钥属性、新 Soc 密钥明文、前 40B 数据的 CRC32 值等
 - HSM 校验前面 40B 数据的 CRC32 值
 - HSM 使用 device root key 加密新 Soc 密钥明文, 对 SoC key 密钥密文 (32B) 计算新的 CRC32 值
 - 写入顺序:
 - 先写 40B 新 key (含密钥属性、新 SoC key 密文、新 CRC32) 到 OTP → 更新 bitmap → 销毁旧 key (设置 destroy)

3. SoC 复位 eHSM

- SoC 收到成功响应后, 通过硬件复位信号触发 eHSM 复位
- HSM 上电后读取 OTP bitmap, 根据 bitmap 修改密钥映射表指向新 SoC Key

4. 轮换完成, BL 和 FW 使用新 SoC key

1.2.3 限制说明

必须由外部管理系统统一保存所有设备的 **RTL SoC KEK**，在外部完成对密钥轮换 mailbox 指令数据加密：

密钥	用途
RTL SoC KEK	加密写入 OTP 的数据，保证终端无法直接获得写入 OTP 的明文数据。

1.2.4 示例

以轮换 SoC Verify Key 为例（`key_type=0`，slot 10），假设 OTP 排布如下（实际由客户决定）：

密钥	当前使用	轮换后使用
SoC Debug Key	slot 9	slot 12
SoC Verify Key	slot 10	slot 6
SoC Encrypt Key	slot 11	slot 7

1. 外部管理系统生成新 SoC Verify Key，加密后经 SoC 传入 HSM（`key_type=0`）
2. HSM 将新 SoC Verify Key 写入 slot 6，更新 bitmap bit0=1
3. Host 复位 HSM，HSM 上电读 bitmap，SoC Verify Key 指向 slot 6，其余不变

实际 OTP 密钥排布和密 钥映射表由客户决定，本文档中的 slot 编号和密钥类型均为示例。

2. OTP 密钥映射表的修改、复用

2.1 原始需求

- 支持 OTP 密钥映射表的修改、复用

2.2 具体方案

2.2.1 改动点

- 根据客户提供的 OTP 密钥映射表，替换、修改 BL/FW 的密钥映射表
- 修改 BL/FW TRM 文档关于密钥映射表的描述

2.2.2 OTP 密钥映射表（客户已确认）

id	name	等级	key 类型	位置	备注	密钥权限
0	Chip root key	0	sym m	OTP- KMU	芯片根密钥	无
1	Device root key	1	sym m	OTP- KMU	设备根密钥	无
2	USER root key	1		OTP- KMU		无
3	eHSM debug/verify key	1	asy mm	OTP- KMU	eHSM 鉴权验 签密钥	无
4	eHSM FW/update verify key	1	asy mm	OTP- KMU	eHSM 镜像验 签密钥	无
5	eHSM FW/update encrypt key	1	sym m	OTP- KMU	eHSM 镜像解 密密钥	无
6	Soc FW/update verify Rotation key	2	asy mm	OTP- KMU		无
7	Soc FW/update encrypt Rotation key	2	sym m	OTP- KMU		无
8	eHSM OTP secret key	2	sym m	OTP- KMU	传输保护密 钥，用于 RAM 密钥的 密文导入导出	1. 密钥可以用于传输（导入/ 导出）时保护密钥（加密 或 MAC）
9	Soc debug verify key	2	asy mm	OTP- KMU	soc 鉴权密钥	无

10	Soc FW/update verify key	2	asy mm	OTP- KMU	soc 镜像验签 密钥	无
11	Soc FW/update encrypt key	2	sym m	OTP- KMU	soc 镜像解密 密钥	无
12	Soc debug verify Rotation key	2	asy mm	OTP- KMU		无
13	UDS	2	asy mm	OTP- KMU	UDS	1. 密钥可以用于签名，或生成 MAC 2. 密钥可以用于验证或验证 MAC 3. 密钥可以用于加密 4. 密钥可以用于解密 5. 密钥可以用于派生或者协商出新密钥
14	Device private Key	2	asy mm	OTP- KMU	DICE 设备私 钥， attestation key	1. 密钥可以用于签名，或生成 MAC 2. 密钥可以用于验证或验证 MAC 3. 密钥可以用于加密 4. 密钥可以用于解密 5. 密钥可以用于派生或者协商出新密钥
15	User auth key	2	asy mm	OTP- KMU		无

6.2 密钥权限

表 13 密钥权限定义

权限名	值	说明
KEY_USAGE_SIGN	0x1	密钥可以用于签名，或生成 MAC
KEY_USAGE_VERIFY	0x2	密钥可以用于验证或验证 MAC
KEY_USAGE_ENCRYPT	0x4	密钥可以用于加密
KEY_USAGE_DECRYPT	0x8	密钥可以用于解密
KEY_USAGE_KEYCREATION	0x80	密钥可以用于派生或者协商出新密钥
KEY_USAGE_CREATION_TRANSP_KEY	0x100	当 KEY_USAGE_KEYCREATION 权限已设置，此权限： • 设置：密钥可以用于创建（派生）传输密钥 • 未设置：密钥可以用于创建（派生）非传输密钥
KEY_USAGE_TRANSPORT	0x400	密钥可以用于传输（导入/导出）时保护密钥（加密或 MAC）

2.2.3 限制说明

- 密钥复用至少包含（已确认，用于支持第 1 个需求的 3 个 SoC 密钥轮换）
 - SoC 升级、启动的 verify key 复用
 - SoC 升级、启动的 encrypt key 复用
 - eHSM 升级、启动的 verify key 复用
 - eHSM 升级、启动的 encrypt key 复用

3. SoC 主动更新 OTP version counter

3.1 原始需求

BL 新增一个 Mailbox 指令（并新增对应的 Host API）：通过指定参数选择更新 SoC/eHSM version counter，且 SoC 端通过指令/API 输入 version counter，以及选择需要更新的 version counter 类型，需要和 DRAM 暂存的 SoC/eHSM version counter 进行比较，一致才允许写入 OTP，都则报错；**FW 暂时不需要支持这个功能**

3.2 具体方案

3.2.1 改动概述

- SoC 调用新增的 MB 指令/Host API 指定参数选择更新 SoC/eHSM version counter，并由 SoC 端输入 version counter

- HSM 会检查输入的 version counter 和 DRAM 暂存的 version counter 是否一致，一致才更新 OTP 中对应的 version counter，否则报错

3.2.2 改动点

- BL 新增 Mailbox 指令适配：
 - 通过指定参数选择更新 SoC/eHSM version counter，并由 SoC 端输入 version counter
- Host 新增 API 适配
 - 和 MB 指令同功能，指定参数选择更新 SoC/eHSM version counter，并由 SoC 端输入 version counte
 - Host TRM 新增 API 描述
- HSM 适配
 - 检查输入的 version counter 和 DRAM 暂存的 version counter 是否一致，一致才更新 OTP 中对应的 version counter，否则报错
- 该指令无生命周期限制

3.2.3 限制说明

- 需要分两次调用指令，分别制定并更新 eHSM、SoC version counter
- FW 原有更新 version counter 代码不做改变（保留原代码），如果在 BL 执行该新增 MB 指令已经更新 version counter，在 FW 启动过程中会因为检查到 DRAM 暂存的版本号和 OTP 版本号一致，不再更新 OTP 中版本号

4. mem ECC 1bit 错误信号改脉冲

4.1 原始需求

mem ECC 1bit 错误是可清除的，可以知道发生了几次

4.2 具体方案

4.2.1 改动点

- 硬件修改 mem ECC 1bit 错误信号由电平改成脉冲，对外表现即为一个脉冲信号
- 外部集成时转成电平供软件查询

4.2.2 限制说明

- 需要外部自己计数，统计发生次数

